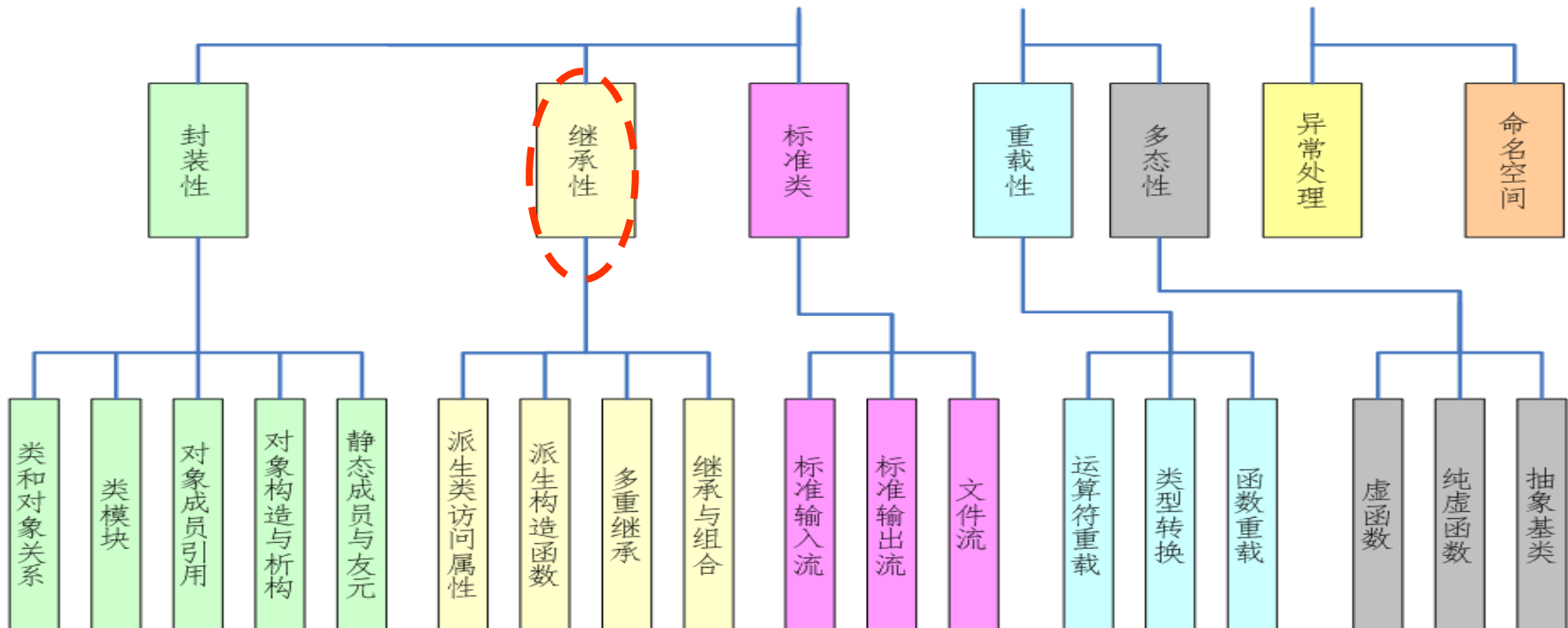


第6讲 继承与派生（上）

- 6.1 继承与派生的概念
- 6.2 派生类的声明和构成
- 6.3 派生类成员的访问属性

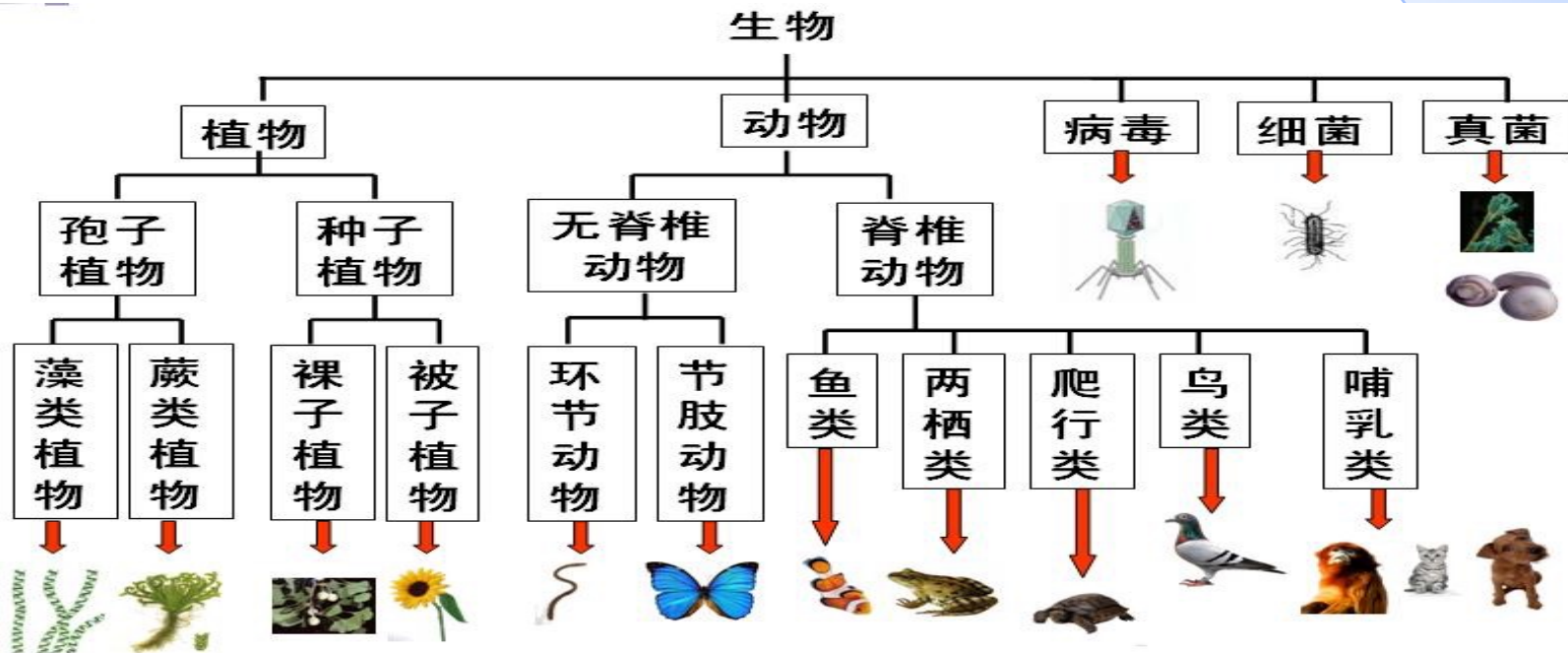
C++的OOP程序 = 对象 + 消息 + 工具



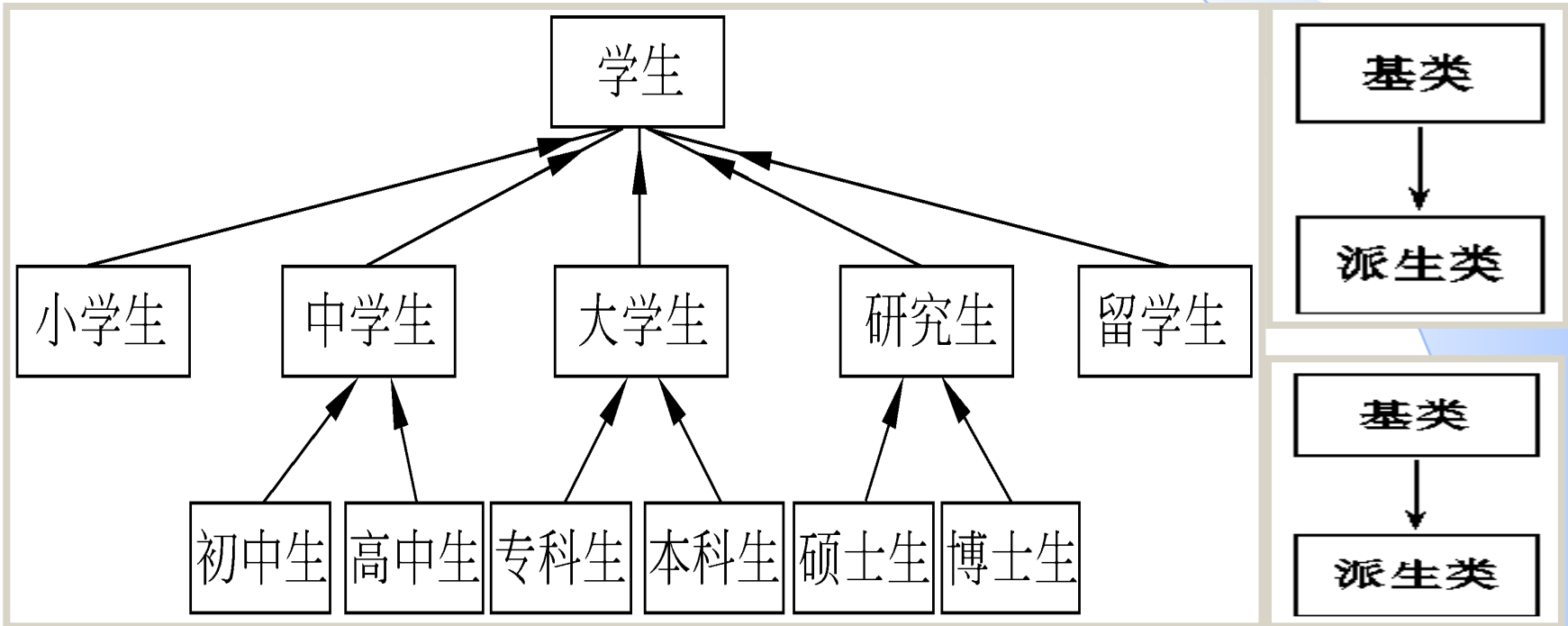
6.1 继承与派生的概念

□ 问题提出？

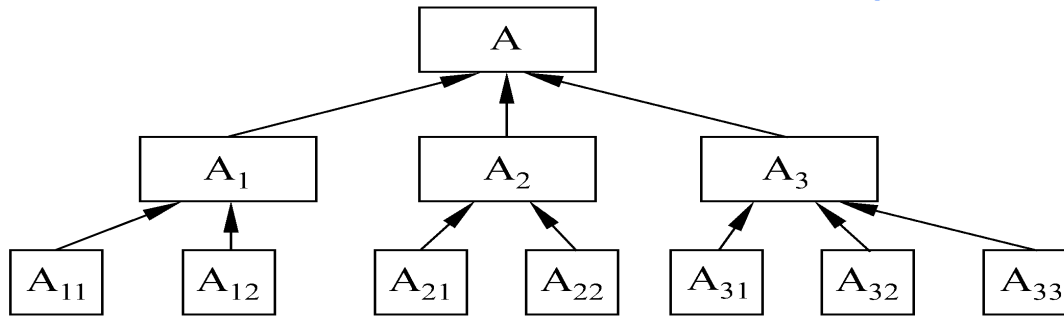
- ✓ 如果类A中包含了若干数据成员和成员函数。新类B中包含了A类中部分相同成员。如何高效地申明新类B呢？
- ✓ C++提供了继承机制，继承(inheritance)体现C++可重用性；
- ✓ 继承法则也体现了大自然的基本规律。



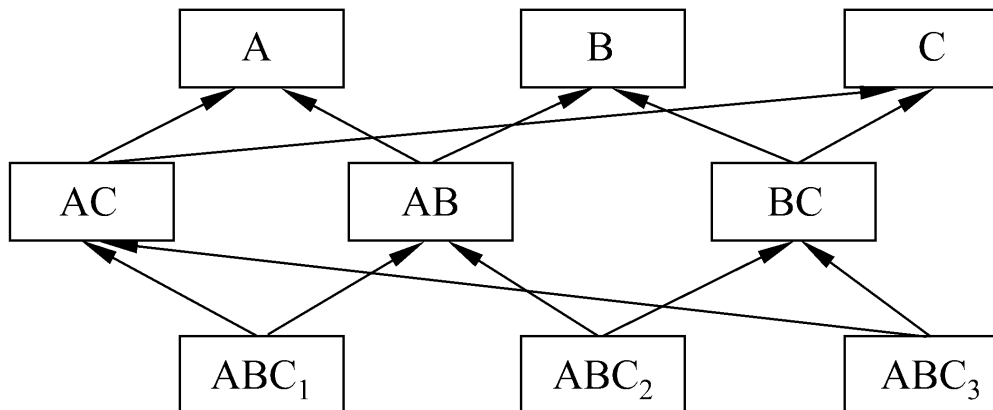
- 类继承: 新类 (派生类) 从已有类 (基类) 获得其特性;
- 类派生: 已有父类 (基类) 产生新的子类 (派生类);
- 一个基类可派生出多个派生类, 每一个派生类又可作为基类再派生出新的派生类, 基类和派生类是相对而言。

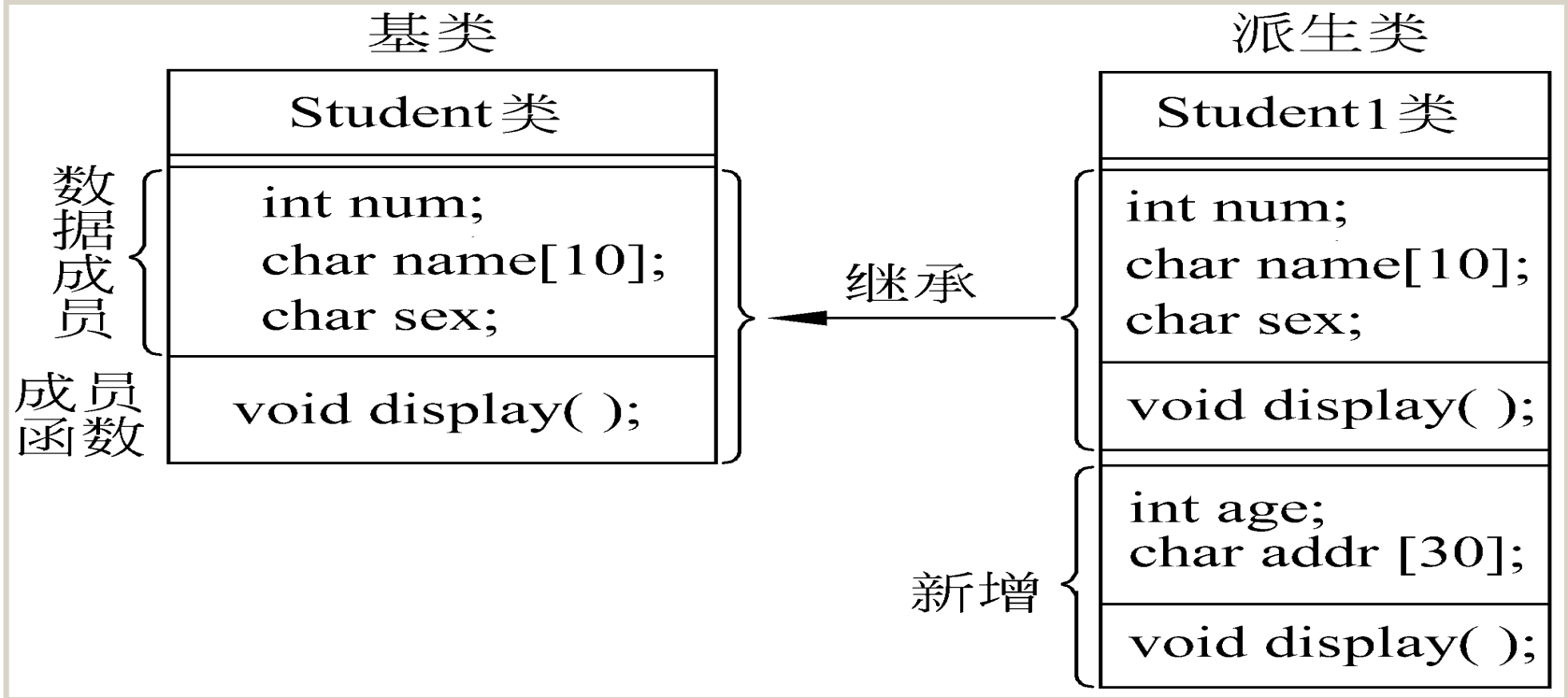


■ 单继承 (single inheritance): 一个派生类只从一个基类派生, 这种继承关系所形成的层次是一个树形结构;



■ 多重继承 (multiple inheritance): 一个派生类可以从多个基类派生;





```

1 class Student1: public Student//声明基类是 Student
2 {public:
3 void display_1() //新增加的成员函数
4 {cout<<"age: "<<age<<endl;
5 cout<<"address: "<<addr<<endl;}
6 private:
7 int age; //新增加的数据成员
8 string addr; }; //新增加的数据成员
9
  
```

```

3 class student
4 {public:
5 void display();
6 private:
7 int num;
8 char name[10];
9 char sex;
10 };
  
```

6.2 派生类的声明和构成

声明派生类的一般形式：

```
class 派生类名： [继承方式] 基类名
{新增加的成员
} ;
```

继承方式： public, private和protected，默认为private

继承性质	基类中成员访问权限	基类成员在派生类中的访问权限的演变
public	public protected private	public protected <u>不可访问</u>
protected	public protected private	protected protected <u>不可访问</u>
private	public protected private	private private <u>不可访问</u>

□ 派生类的构造

- (1) 遗传：从基类接收成员。派生类把基类全部的成员（不包括构造函数和析构函数）接收过来；
- (2) 变异：在声明派生类时改变基类访问属性；增加新成；同名屏蔽；
- (3) 派生类的构造函数和析构函数。构造函数和析构函数是不能从基类继承，另外定义。

□ 面向对象程序设计核心思想之一

- (1) 声明一个抽象基类，只提供某些最基本的功能；
- (2) 在声明派生类时加入某些具体的功能，形成适用于某一特定应用的派生类。

6.3 派生类成员的访问属性

□ 公用继承（public）的派生类访问属性调整

表 11.1 公用基类在派生类中的访问属性

公用基类的成员	在公用派生类中的访问属性
私有成员	不可访问 可类外、类内访问
公用成员	公用
保护成员	保护 只能类内访问

□ 派生类成员的访问方式

- (1) 类内: 在派生类内可直接访问基类public、protected成员;
- (2) 类外: 在派生类外（通过对象）能访问基类的public成员;
- (3) 派生类新增成员按照访问属性来限定。

```
1. class Employee {
2.     String name; //name是私有成员，在一切派生类中均不可见
3.     Date birthday;
4.     char sex;
5.     short department;
6. public:
7.     Employee();
8.     String getName() const { return name; }
9.     void Print() const
10.    { cout << name << " is an employee" << endl; }
11.};
12.class Manager : public Employee {
13. public:
14.     Manager();
15.     void Print() const //覆盖基类同名同类型成员函数
16.     {
17.         cout << name << "is a manager" << endl; //错误
18.         cout << getName() << " is a manager" << endl; //正确
19.     }
20.};
```

```

1. class Employee {
2. protected:
3.     String name; //name是保护成员，向一切派生类开放
4.     Date birthday;
5.     char sex;
6.     short department;
7. public:
8.     Employee();
9.     void Print() const
10.    { cout << name << " is an employee" << endl; }
11. };

12. class Manager : public Employee {
13. public:
14.     Manager();
15.     void Print() const
16.     { cout << name << "is a manager" << endl; //正确
17.     }
18. }

```

```

1. class Employee {
2.     String name;
3.     Date birthday;
4.     char sex;
5.     short department;
6. public:
7.     Employee();
8.     void Print() const
9.     { cout << name << endl; }
10. };

```

```

11. class Manager : public Employee {
12.     short level;
13. public:
14.     Manager();
15.     void Print() const
16.     { Employee::Print();
17.         cout << level << endl;
18.     }
19. }

```

```

void main()
{
    Employee e1;
    Manager m1;
    ...
    e1.Print();
    m1.Print();
}

```

必须用Employee::表明调用基类的Print()函数，若直接写Print()将调用派生类的Print()函数自己！

//打印Employee部分的信息
//打印Manager特有信息

例. 访问公有基类的成员(指出是否错误)

```

1  Class Student//声明基类
2  {public:                                //基类公用成员
3  void get_value( )
4  {cin>>num>>name>>sex;}
5  void display( )
6  {cout<<" num: "<<num<<endl;
7  cout<<" name: "<<name<<endl;
8  cout<<" sex: "<<sex<<endl;}
9  private:                               //基类私有成员
10     int num;
11     string name;
12     char sex;};
13
14  class Student1: public Student          //以public方式声明派生类Student1
15  {public:
16  void display_1( )
17  { cout<<" num: "<<num<<endl;           //引用基类的私有成员, ?
18    cout<<" name:"<<name<<endl;         //引用基类的私有成员, ?
19    cout<<" sex:"<<sex<<endl;           //引用基类的私有成员, ?
20    cout<<"age:"<<age<<endl;           //引用派生类的私有成员, ?
21    cout<<"address:"<<addr<<endl;}     //引用派生类的私有成员, ?
22  private:
23     int age;
24     string addr;};

```

```

1  □ Class Student//声明基类
2  {public:                                //基类公用成员
3  void get_value( )
4  {cin>>num>>name>>sex;}
5  void display( )
6  {cout<<" num: "<<num<<endl;
7  cout<<" name: "<<name<<endl;
8  cout<<" sex: "<<sex<<endl;}
9  private:                                //基类私有成员
10     int num;
11     string name;
12     char sex;};
13
14 □ class Student1: public Student        //以public方式声明派生类Student1
15 {public:
16 □ void display_1( )
17     { cout<<" num: "<<num<<endl;        //企图引用基类的私有成员, 错误
18       cout<<" name:"<<name<<endl;      //企图引用基类的私有成员, 错误
19       cout<<" sex:"<<sex<<endl;        //企图引用基类的私有成员, 错误
20       cout<<" age:"<<age<<endl;        //引用派生类的私有成员, 正确
21       cout<<" address:"<<addr<<endl;}  //引用派生类的私有成员, 正确
22 private:
23     int age;
24     string addr;};

```

```

1 #include "string"
2 #include "iostream"
3 using namespace std;
4 class Student//声明基类
5 {public:                                //基类公用成员
6 void get_value()
7 {cin>>num>>name>>sex;}
8 void display()
9 {cout<<"num:"<<num<<endl;
10 cout<<"name:"<<name<<endl;
11 cout<<"sex:"<<sex<<endl;}
12 private:                             //基类私有成员
13 int num;
14 string name;
15 char sex;};
16 class Student1: public Student//以public方式声明派生类Student1
17 {public:
18 void display_1()
19 {cout<<"age:"<<age<<endl;           //引用派生类的私有成员，正确
20 cout<<"address:"<<addr<<endl;}      //引用派生类的私有成员，正确
21 private:
22 int age;
23 string addr;};
24 int main()
25 {Student1 stud; //定义派生类Student1的对象stud
26 stud.display(); //调用基类的公用成员函数，输出基类中3个数据成员的值
27 stud.display_1(); //调用派生类的公用成员函数，输出派生类中两个数据成员的值
28 return 0;}

```

C:\WINDOWS\system32\cmd.exe

```

num:1458808
name:
sex:
age:1241948
address:
请按任意键继续. . .

```

6.3 派生类成员的访问属性

□ 私有继承 (private) 派生类访问属性调整

表 11.2 私有基类在派生类中的访问属性

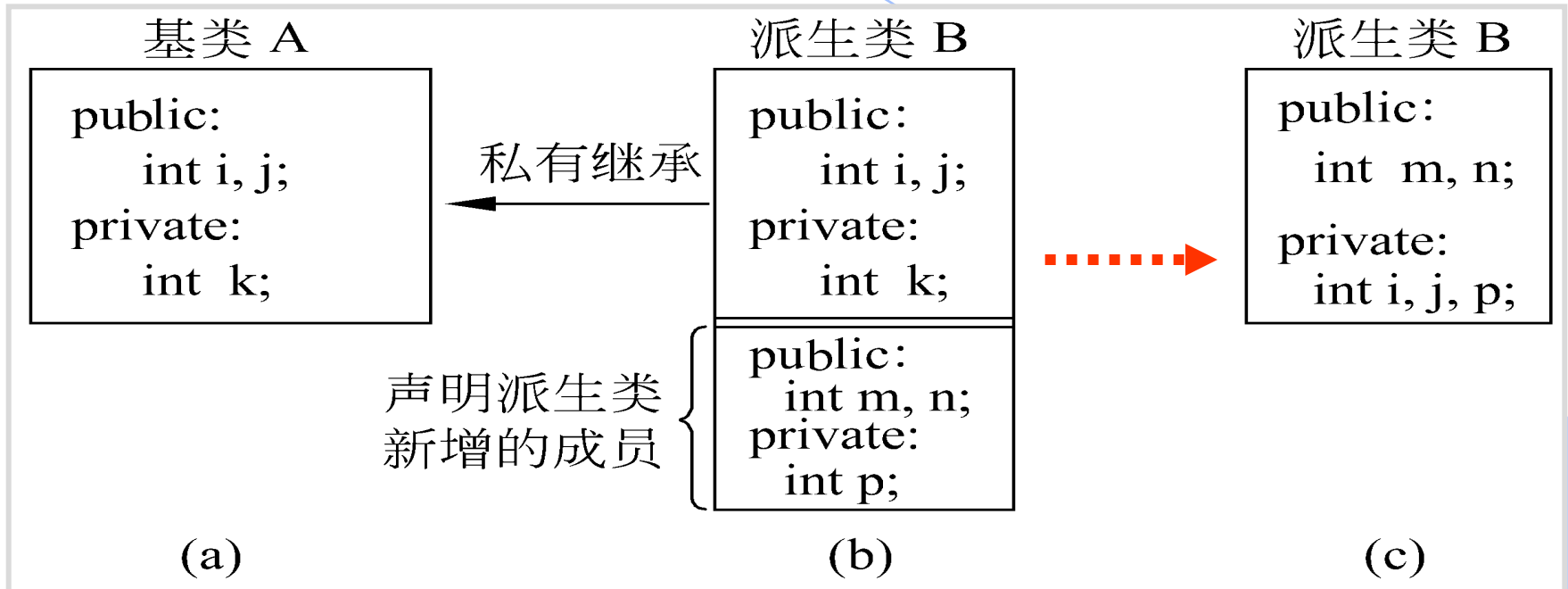
私有基类中的成员	在私有派生类中的访问属性
私有成员	不可访问
公用成员	私有
保护成员	私有

只能类内访问

□ 派生类成员的访问方式

- (1) 类内: 在派生类内可访问基类的公用成员和保护成员;
- (2) 类外: 在派生类外 (通过派生类对象) 不能访问基类的任何成员;
- (3) 派生类新增成员按 public、protected、private 访问规则。

□ 私有继承的派生类中访问属性变异



□ 程序设计思想之二

采用私有继承可对于基类中不需要再往下继承的功能把它**隐蔽**起来，这样下一层派生类无法访问基类的任何成员；

```

1 #include "string"
2 #include "iostream"
3 using namespace std;
4 class Student//声明基类
5 {public:
6 void get_value()
7 {cin>>num>>name>>sex;}
8 void display()
9 {cout<<"num:"<<num<<endl;
10 cout<<"name:"<<name<<endl;
11 cout<<"sex:"<<sex<<endl;}
12 private:
13 int num;
14 string name;
15 char sex;};
16 class Student1: private Student//用私有继承方式声明派生类Student1
17 {public:
18 void display_1() //输出两个数据成员的值
19 {cout<<"age:"<<age<<endl; //引用派生类的私有成员，正确
20 cout<<"address:"<<addr<<endl;} //引用派生类的私有成员，正确
21 private:
22 int age;
23 string addr;
24 };
25 int main()
26 {Student1 stud1; //定义一个Student1类的对象stud1
27 stud1.display(); //?, 私有基类的公用成员函数在派生类中是私有函数
28 stud1.display_1(); //?, Display_1函数是Student1类的公用函数
29 stud1.age=18; //?,
30 return 0; }

```

输出

显示以下输出 (S): 生成

```

1>----- 已启动全部重新生成: 项目: test2, 配置: Debug Win32 -----
1>正在删除项目 "test2" (配置 "Debug|Win32") 的中间文件和输出文件
1>正在编译...
1>1. cpp
1>. \1. cpp (27) : error C2247: 'Student::display' not accessible because 'Student1' uses 'private' to inh
1>. \1. cpp (8) : see declaration of 'Student::display'
1>. \1. cpp (16) : see declaration of 'Student1'
1>. \1. cpp (5) : see declaration of 'Student'
1>. \1. cpp (29) : error C2248: 'Student1::age' : cannot access private member declared in class 'Student1'
1>. \1. cpp (22) : see declaration of 'Student1::age'
1>. \1. cpp (16) : see declaration of 'Student1'
1>生成日志保存在 "file://e:\vc-exercise\test2\test2\Debug\BuildLog.htm"
1>test2 - 2 个错误, 0 个警告
===== 全部重新生成: 0 已成功, 1 已失败, 0 已跳过 =====

```

```

1 #include "string"
2 #include "iostream"
3 using namespace std;
4 class Student//声明基类
5 {public:
6 void get_value( )
7 {cin>>num>>name>>sex;}
8 void display( )
9 {cout<<"num:"<<num<<endl;
10 cout<<"name:"<<name<<endl;
11 cout<<"sex:"<<sex<<endl;}
12 private :
13     int num;
14     string name;
15     char sex;};
16 class Student1: private Student//用私有继承方式声明派生类Student1
17 {public:
18 void display_1() //输出5个数据成员的值
19 {display(); //调用基类的公用成员函数，输出3个数据成员的值
20 cout<<"age:"<<age<<endl; //引用派生类的私有成员，正确
21 cout<<"address:"<<addr<<endl;} //引用派生类的私有成员，正确
22 private:
23     int age;
24     string addr;};
25
26 int main( )
27 {Student1 stud1;
28 stud1.display_1();//display_1函数是派生类Student1类的公用函数
29 return 0;}

```

//基类私有成员

C:\WINDOWS\system32\cmd.exe

```

num:1458808
name:
sex:
age:1241948
address:
请按任意键继续. . .

```

□ 程序设计思想之三

在派生类中可用 `using 基类::成员` 恢复基类防控级别；

- (1) 基类 `private` 成员不能在派生类中任何地方用 `using` 声明；
- (2) 基类 `protected` 成员可在派生类中任何地方用 `using` 声明；
当在 `public` 下声明时，可派生类外（通过对象）访问该成员，但不能用基类对象访问该成员；当在 `protected` 下声明时，该成员可以被继续派生下去；当在 `private` 下声明时，不能载派生类外（通过对象）访问；
- (3) 基类 `public` 成员，可在派生类中任何地方用 `using` 声明。
当在 `public` 下声明时，可派生类外（通过对象）访问该成员，但不能用基类对象访问该成员；当在 `protected` 下声明时，该成员可以被继续派生下去；当在 `private` 下声明时，不能载派生类外（通过对象）访问。

Class CAnimal

{public:

void GetWeight();

Private:

float m_fweight;};

Class CPig:private CAnimal

{public:

using CAnimal::GetWeight;

void GetPorkWeight();

Private:

Float m_fPorkWeight;};

#include <iostream>

Using namespace std;

void main(void)

{

Cpig apig;

apig.GetWeight();

apig.GetPorkWeight()

;

}

```

1  class Base
2  {
3  protected:
4      void test1() { cout << "test1" << endl; }
5      void test1(int a) {cout << "test2" << endl; }
6
7      int value = 55;
8  };
9
10 class Derived : Base    //使用默认继承
11 {
12 public:
13     //using Base::test1;    //using只是声明, 不参与形参的指定
14     //using Base::value;
15     void test2() { cout << "value is " << value << endl; }
16 };

```

问题：分析上述编译会出错吗？为什么？怎么纠错？

```

2  #include "pch.h"
3  #include <iostream>
4
5  #include "string"
6  using namespace std;
7  class tester
8  {
9  public:
10     tester() : i(5), ch('x') {};
11 private:
12     int i;
13     char ch;
14 };
15 class tester1 : public tester
16 {
17 private:
18     int y = 0;
19 };
20
21 int main(void)
22 {
23     tester1 myTester1;
24     char* p = NULL;
25     p = (char*)&myTester1 + sizeof(int);
26     cout << "Address of ch = " << (void*)p << endl;
27     cout << "ch = " << *(p) << endl;
28     //getchar();
29     *p = 'y';
30     cout << "Now ch = " << *(p) << endl;
31     return 0;
32 }

```

C:\Users\hp\source\repos\Con

Address of ch = 0136FDF0
ch = x

Address of ch = 010FFC60

ch = x
Now ch = y

C:\Users\hp\source\repos\ConsoleApplicati
若要在调试停止时自动关闭控制台，请启用“工
按任意键关闭此窗口...

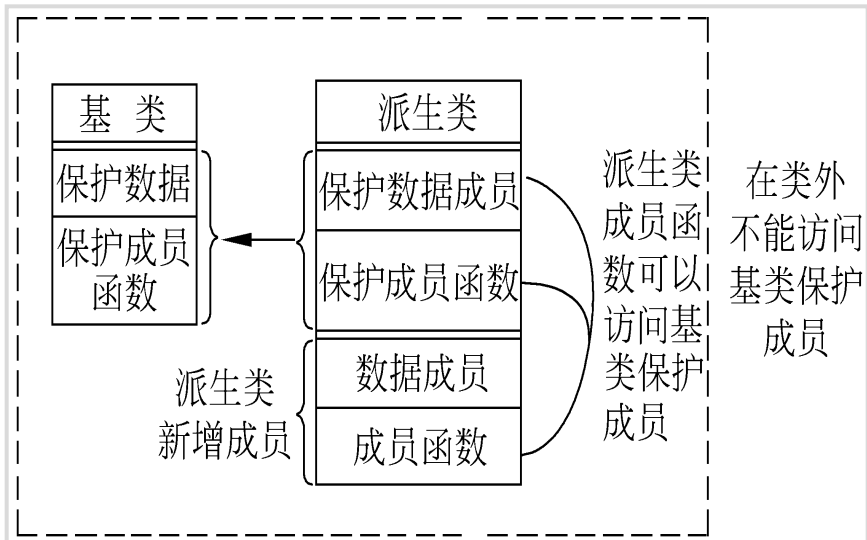
6.3 派生类成员的访问属性

■ **保护成员** (protected) : 在访问属性等价于私有 (Private) 成员；但在继承方式区别于 Private 成员。

□ 保护继承 (protected) 在派生类的访问属性调整

表 11.3 基类成员在派生类中的访问属性

基类中的成员	在公用派生类中的访问属性	在私有派生类中的访问属性	在保护派生类中的访问属性
私有成员	不可访问	不可访问	不可访问
公用成员	公用	私有	保护
保护成员	保护	私有	保护



□ **protected** 派生类访问方式

- (1) **类内**: 在派生类内可访问基类公用和保护成员；
- (2) **类外**: 在派生类外（用对象）不能访问基类任何成员；
- (3) **新增成员**按访问属性规定。


```

1. class Employee {
2. protected:
3.     String name;
4.     Date birthday;
5.     char sex;
6.     short department;
7. public:
8.     Employee();
9.     void Print() const
10.    { cout << name << "is an employee" << endl; }
11. };

```

```

12. class Manager : protected Employee {
13.     short level;
14. public:
15.     Manager();
16.     void Print() const
17.     { //派生类可以访问基类的保护成员
18.         cout << name << "is a manager" << endl;
19.     }
20. }

```

```

void main()
{
    Employee e1;
    Manager m1;
    ...
    m1.Print();
    cout << m1.name; //错误,
                     //外界不能访问保护成员
}

```

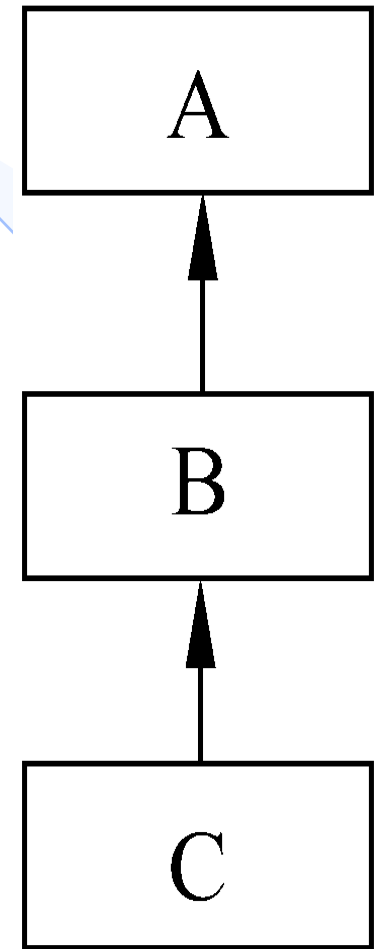
6.3 派生类成员的访问属性

□ 多级派生时的访问属性

(1) 如果有下图所示的派生关系：类A为基类，类B是类A的派生类；类C是类B的派生类，则类C也是类A的派生类；

(2) 类B称为类A的直接派生类，类C称为类A的间接派生类。类A是类B的直接基类，是类C的间接基类；

(3) 在多级派生下，各成员的访问属性仍按前面的原则确定。



```

class A           //基类
{public:
  int i;
protected:
  void f2();
  int j;
private:
  int k;};

class B: public A //public方式
{public:
  void f3();
protected:
  void f4();
private:
  int m;};

class C: protected B //protected方式
{public:
  void f5();
private:
  int n;};
  
```

	i	f2	j	k	f3	f4	m	f5	n
基类A	公用	保护	保护	私有					
公用派生类B	公用	保护	保护	不可访问	公用	保护	私有		
保护派生类C	保护	保护	保护	不可访问	保护	保护	不可访问	公用	私有

26

- 无论何种继承，派生类中都不能访问基类的私有成员；
- 如多级派生时都采用公用继承，那么直到最后一级派生类都能访问基类的公用成员和保护成员。

27

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4 class Student//声明基类
5 {public:                                //基类公用成员
6     void display();
7     protected:                        //基类保护成员
8     int num;
9     string name;
10    char sex;};
11
12 void Student::display()                //定义基类成员函数
13 {cout<<"num:"<<num<<endl;
14  cout<<"name:"<<name<<endl;
15  cout<<"sex:"<<sex<<endl;}
16
17 class Student1: protected Student    //用protected方式声明派生类Student1
18 {public:
19     void display1();                  //派生类公用成员函数
20     private:
21     int age;                          //派生类私有数据成员
22     string addr;                      //派生类私有数据成员
23
24     void Student1::display1()         //定义派生类公用成员函数
25     {cout<<"num: "<<num<<endl;        //引用基类的保护成员, ?
26      cout<<"name:"<<name<<endl;      //引用基类的保护成员, ?
27      cout<<"sex:"<<sex<<endl;        //引用基类的保护成员, ?
28      cout<<"age:"<<age<<endl;        //引用派生类的私有成员, ?
29      cout<<"address:"<<addr<<endl; } //引用派生类的私有成员, ?
30
31 int main()
32 {Student1 stud1;                      //stud1是派生类Student1类的对象
33  stud1.display1();                    //?, display1是派生类中的公用成员函数
34  // stud1.num=10023;                  //?, 外界不能访问保护成员
35  return 0;

```

C:\WINDOWS\system32\cmd.exe

```

num: 1458808
name:
sex:
age:1241948
address:
请按任意键继续. . .

```

本讲重点分析

- ❑ 派生类声明形式: class 派生类名: [继承方式] 基类名
- ❑ 继承方式: public, protected和private
- ❑ 类成员访问属性
 - ✓ 基类成员: public, protected, private
 - ✓ 派生类成员: public, protected, private 和不可访问
- ❑ 派生类的成员访问方式
 - ✓ 类新成员访问: public, protected, private
 - ✓ 类内对基类继承成员访问: public, protected
 - ✓ 类外(通过对象)对基类继承成员: public

第6次作业（必做题）

第1题：教材本章后面习题中的第8题。阅读程序，分析程序运行结果，并回答问题。

第2题：在上次（第5次）作业的第1题的基础上，从People(人员)类派生出student(学生)类，添加属性：班号char classNo[7]；从People(人员)类派生出teacher(教师)类，添加属性：职务char principalship[11]、部门char department[21]；从student类中派生出graduate(研究生)类、添加属性：专业char subject[21]、导师teacher advisor；从teacher类和graduate类派生出TA(助教)类，添加属性：RA。要求编制能管理上述四类人员的程序，并实现数据项录入、显示操作。

选做题

1. 参考教材本章综合程序的代码，在前面作业基础上进行修改：要求采用继承方式，新增员工设岗，设立3类岗位：经理（manager）1名，技术（technician）岗10名和其余为销售岗位（salesman）。岗位不同，待遇不同。经理拿固定月薪12000元，技术人员按每小时260拿月薪，销售岗按当月的销售额提成5%。新增加类属性可以自由添加；其他细节部分自由设计。鼓励创新！！！！

2. 上机调试本PPT 中的第23页中的程序。如果将类tester中的数据成员i改为：long int. 那如何实现现有程序的功能？

笑话一则：小明为啥在起跑线上？

- 妈妈：“小明，要是你不再努力学习，小学学习基础没打好，你就会输在人生起跑点上！”
- 小明：“我早就输在起人生跑点上啦。”
- 妈妈：“你输了什么？”
- 小明：“继承与遗传呀”

因此，同学们应该感恩“好的基类”；同时注意设计“好的基类”。